

上海财经大学本科毕业论文（初稿）

面向金融问答的时点约束义务感知检 索规划研究

学院： 金融学院

专业： 金融学（数据与智能方向）

作者： 匿名

指导教师： 待定

2026 年 9 月

摘要

金融问答并不只是从语料库中找出最相关的一句话。一个问题可能同时要求多个实体、财务年度、季度以及申报类型，并且只有在问题时点之前公开的文件才具备证据资格。本文提出 FinPlan-RAG，一种非预言式的时点约束义务感知检索规划方法。系统将问题解析为 (, , ,) 四元组义务，针对尚未解决的义务逐项检索，在固定查询预算下仅接纳时点有效的文档，并在证据闭合前保持保守状态。

本文首先说明研究缺口：单一相关性分数不能保证停止决策所需的证据集合已经闭合。随后给出两个世界具有相同可见分数但需要相反动作的构造性证明，并证明去掉时点过滤后可以发生未来信息泄漏。在本地可复现实验中，我们使用 20 个随机种子、7 类场景、每类 40 个样本和统一预算 10 进行 5,600 个 case 的机制检验。FinPlan-RAG 的闭合率为 0.366，而 single-shot 基线为 0.101；平均查询次数为 8.87，而基线为 10。去掉反证模块后闭合率上升至 0.511，但反证召回率由 0.605 降至 0.315，过度断言率由 0.013 升至 0.035；去掉时点过滤后未来文档泄漏率达到 0.502。上述结果仅支持受控机制结论，不构成真实财报问答或顶刊 SOTA 结论。

本文同时提供面向 Codex、Claude Code 等代码 Agent 的统一 JSON 工具协议和标准输入输出适配器。研究结果、协议哈希和测试命令均在代码仓库中公开，以便后续使用冻结的金融文件数据、共享阅读器和强基线进行独立复核。

关键词：金融问答；检索增强生成；时点约束；义务状态；证据闭合；反证；代码 Agent

目录

第一章 绪论

1.1 研究背景

金融披露具有明确的实体、年度、季度、申报类型和公开时间。传统 RAG 通常将检索看作排序问题，返回若干高相关段落，却没有显式表示“还缺少哪一份申报文件”。这种隐含状态会造成遗漏第二个实体或期间、把后续修订文件带入历史问题、以及忽视限制性披露等风险。

1.2 研究目标与假设

本文检验：在信息、阅读器和查询预算相同的条件下，显式义务状态是否能够改善下一步检索和停止决策。核心假设 H1 为：义务感知状态提高证据闭合率并减少无效查询；显式反证和时点过滤牺牲部分覆盖率，但降低过度断言和未来信息泄漏。

1.3 研究缺口与贡献

现有相关性排序、稀疏/稠密检索和长上下文记忆基准解决了“哪些文本相似”问题，但没有保证“所有被问题要求的文件都已被覆盖”。本文形式化金融检索义务，给出相关性分数不足以支撑停止决策的证明，实现非预言式规划器，开展基线、消融及时点压力测试，并提供代码 Agent 统一接口。

第二章 理论基础与研究缺口证明

2.1 义务与闭合

对问题 q ，令 $O(q)$ 为义务集合，义务元素记为 $o = (e, y, f, p)$ ；令 $D_{\leq c}$ 表示公开时间不晚于截止时点 c 的文件。

定义 2.1 (证据闭合). 当每个具有已知文档编号的义务都至少被一个时点有效文档覆盖，且所需字段的投票结果达到解析阈值时，称证据状态闭合。闭合是检索过程性质，不等于答案正确。

命题 2.1 (同分数反例). 若存在两个潜在世界具有相同可见历史和分数，但对未解决义务的最优动作相反，则任意仅依赖该分数的随机规则都具有非零最坏世界后悔。设正、负世界的动作边际分别为 m_+ 和 m_- ，规则以概率 p 选择正动作，则

$$\max\{(1-p)m_+, pm_-\} \geq \frac{m_+m_-}{m_+ + m_-}$$

证明：正世界选择负动作的概率为 $1-p$ ，负世界选择正动作的概率为 p ；平衡两项损失下界即得结论。

2.2 时点安全性

定理 2.1 (未来信息过滤). 若规划器只将 $available_at \leq c$ 的片段写入证据状态，则输出上下文不包含未来文件。若去掉该谓词，则可以构造未来文件相关性更高而被最大分数选择消费的 *case*。

第三章 FinPlan-RAG 算法设计

3.1 总体框架

系统流程为：问题解析 → 义务生成 → 时点过滤 → 义务定向 BM25 检索 → 证据状态聚合 → 闭合检查 → 阅读器回答或拒答。数据结构包括 DocumentObligation、Chunk、RetrievalResult 和 EvidenceState。

3.2 期间解析与义务生成

系统将 Q1-Q3 映射为 10-Q，将 Q4、FY 和 annual 映射为 10-K/FY；“上半年”映射为 Q2，“下半年”映射为 Q3 与 Q4。解析结果与可见申报元数据求交，输出稳定排序的义务元组。

3.3 检索与停止

对每个未解决义务 o ，构造 $q \oplus o$ 查询，并要求片段包含实体和年份词。BM25 只从截止时点之前的片段中选择一个最高分结果。状态写为

$$E_t = (O(q), C_t, n_t, closed_t, F_t)$$

其中 C_t 为已覆盖文档集合， n_t 为查询次数， F_t 为未来文档诊断集合。闭合前阅读器可以拒答或请求继续检索，不能把闭合当作答案正确性的替代品。

3.4 代码 Agent 接口

仓库新增 `finplan-rag-agent --schema` 命令，输出 OpenAI-compatible function schema；默认模式按 JSONL 读取请求并输出 JSONL 响应。响应包含义务、覆盖文档、查询次数、闭合标记、未来文档诊断以及 `answer_with_reader` 或 `abstain_or_retrieve_more` 建议。

第四章 实验设计与结果

4.1 实验协议

受控实验使用 20 个种子、7 个场景、每场景每种子 40 个 case，总计 5,600 个 case，预算为 10。所有方法共享路径、文件、截止时点、解析器和预算。效用函数预先固定为：正确 = 1、拒答 = -0.15、其他错误 = -1、过度断言额外 = -1、每次查询 = -0.03。完整结果文件的 SHA-256 为 E5D207A2...42CAA28。

4.2 主结果

表 4.1: 受控机制实验均值。除过度断言、查询次数和未来泄漏外，数值越大越好。

方法	准确率	闭合率	过度断言	反证召回	查询数	效用	未来泄漏
Single-shot	.066	.101	.004	.651	10.00	-.407	.000
固定分解	.175	.293	.014	.658	10.00	-.363	.000
通用自适应	.193	.343	.015	.632	10.00	-.371	.000
FinPlan 状态	.191	.366	.013	.605	8.87	-.358	.000

相对 single-shot，FinPlan 的闭合率提高 0.2645，查询次数减少 1.1279。与通用自适应相比，准确率差异为 -0.0020，说明当前证据主要支持流程和停止改进，而非普遍答案准确率优势。

4.3 消融与压力测试

去掉反证后闭合率为 0.511，查询次数为 7.92，但反证召回由 0.605 降为 0.315，过度断言由 0.013 升为 0.035。去掉自适应停止后始终使用 10 次查询；去掉时点过滤后未来文档泄漏率为 0.502，效用降至 -0.438。

第五章 讨论与局限

实验只支持受控机制层结论。合成世界不能替代真实 SEC/交易所文件，闭合率不能替代答案正确率，BM25 也不能代表强稠密检索器。当前实现没有声称训练新模型、没有把 SQCAD 的结果当作 FinPlan 的结果，也没有声称顶刊 SOTA。下一阶段应冻结真实财报数据、共享阅读器、时间切分和强基线，并报告答案正确率、证据闭合、未来泄漏、过度断言和成本的联合 Pareto 前沿。

第六章 结论

本文将金融 RAG 重新表述为截止时点下的义务完成问题。理论反例表明，仅凭相关性分数无法保证停止决策；实现与受控实验显示，显式状态提高闭合并减少查询，但反证和时点约束会引入可测成本。代码 Agent 适配器使 Codex、Claude Code 等工具能够在同一可审计契约下调用系统。

参考文献

参考文献

- [1] Lewis, P. 等. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS, 2020.
- [2] Maharana, A. 等. LoCoMo: Long-context conversation memory benchmark. ACL, 2024.
- [3] Wu, L. 等. LongMemEval: Benchmarking long-term memory of large language models. arXiv, 2024.

附录：证据与复现清单

表中数字来自 `paper/results/controlled_summary.json`, 源文件哈希记录在同目录 README。运行命令为 `PYTHONPATH=src python -m finplan_rag.controlled_planning --seeds 20 --cases-per-world 40 --budget 10`; 测试命令为 `python -m pytest -q`。大规模语料、模型权重和真实数据遵循仓库 `DATA_STORAGE.md`, 不写入 Git。